

Response to Reviewers

Manuscript: 48aab7a5-1e5a-4311-bc0c-190acc3d8ee4

Workload-Aware Data Lakehouse Maintenance Using Hierarchical Deep Reinforcement Learning
Scientific Reports

Preamble

We thank the editor and both reviewers for a set of comments that were, in our assessment, correct on every substantive point. Acting on them did not merely add supporting evidence — it changed several of our conclusions.

In the course of building the multi-seed protocol that Reviewer 1 (comment 1) and Reviewer 2 (comment 1) requested, we discovered **two defects in our own offline training pipeline** that had materially affected the published results. We report these in full in a new subsection (*Corrections to the training pipeline*) and summarise them here, because they are the reason several headline numbers have changed:

1. **Train/serve normalisation skew.** Training z-scored `rows_ingested` and `ingestion_rate` per dataset, while the deployed agent used fixed constants. Inference features were therefore 10–50× outside the training range, saturating the network into an effectively **constant COMPACT_128KB policy**. The adaptivity reported in the original submission was an out-of-distribution artefact, not learned behaviour.
2. **Trajectory-identity leakage.** The staleness features (`steps_since_compact`, `steps_since_partition_change`) encode *which heuristic generated a trajectory* — near zero for `AlwaysCompact`, unbounded for `No_Maintenance`. Advantage-weighted regression therefore cloned the generating policy rather than reading table state. At a fixed table state, $P(\text{NOOP})$ rises from 0.00 to 0.94 as `steps_since_compact` goes from 0 to 30.

Both are corrected. **The environment itself is unchanged**, and it reproduces the published heuristic baselines to within 0.003 (e.g. `AlwaysCompact_C128` 0.1869 → 0.1870), which establishes that the changed RL numbers are attributable to the pipeline corrections rather than to environment drift.

Consequences for the paper’s claims.

Four claims did not survive and have been removed or rewritten: the architecture ranking, the attention advantage, the DDQN adaptation collapse, and “all three agents outperform 17 baselines”. The central claim — that hierarchical RL outperforms hand-tuned heuristics — *does* survive, and is now supported by a paired significance test with a large effect size and consistency across all five independent training runs.

Scale of the new experiments.

216 seeded evaluation runs (plus 16 online-adaptation runs), all validated for data integrity, each carrying a manifest recording seed, configuration, git commit, hardware and wall-clock time.

Response to the Editor

Comment E-1. *Please ensure the results are accurately reported, any overstated conclusions are rewritten and the limitations of the work fully explained.*

Response: This is the organising principle of the revision. Concretely:

- Every performance claim now carries a bootstrap 95 % confidence interval and, where a comparison is made, a paired non-parametric test with an effect size and Holm–Bonferroni correction.
- Claims that do not survive testing have been **removed rather than softened** (details under R1.1, R2.2, R2.3, R2.11).
- A substantially expanded *Limitations* section now states: the micro-scale of the simulation, the absence of a real production query engine, the fact that the pruning matrix is assumed rather than measured, the train/eval objective mismatch, the statistical power limits of individual sub-studies, and the fact that RL is **not** the cheapest policy in our own cost model.

Comment E-2. *The underlying code must be deposited in a recognised DOI-assigning repository (e.g. Zenodo) and linked from Methods or a Code Availability section.*

Response: Done. The code, the simulation environment, all model weights and the complete experimental data are archived on Zenodo at

DOI: 10.5281/zenodo.21860273

(release v1.0-revision, archived from

<https://github.com/OmarEssameldinMousa/LakePilot-Research-prototype>)

The deposit carries an MIT licence, an environment pinned to the exact package versions used for every reported result (PySpark 3.5.7, TensorFlow 2.20.0, NumPy 2.2.6, pandas 2.3.3), end-to-end reproduction instructions, a `manifest.json` per run, and the data-integrity validator used to audit every run. A new **Code availability** section cites the DOI, and the Data availability statement has been updated to cite it alongside the corrected repository URL.

We also note that the reviewer was correct that the repository URL in the original Data Availability statement was **truncated and did not resolve** (`.../LakePilot-Research-prot` returns HTTP 404; the complete URL returns 200). This has been corrected — see R2.17.

Reviewer #1

Comment R1.1 — Statistical validation. *How can readers be confident the reported improvements are not due to random seed or initialization variance? Please provide confidence intervals, bootstrap estimates, or non-parametric tests across multiple independent training runs.*

Response: Accepted in full, and the answer changed our conclusions.

We retrained all three architectures with **five independent random seeds** and evaluated every checkpoint frozen on two protocols. Environment seeds are a deterministic function of (protocol, seed slot, episode) and **not** of the policy, so every policy sees identical episodes and all comparisons are *paired*. We report percentile bootstrap 95 % CIs (10,000 resamples), Wilcoxon signed-rank **and** paired permutation tests, Cliff’s delta, and Holm–Bonferroni correction across each comparison family.

New Table 1 (500-step protocol, 25 matched episodes per policy):

Policy	Reward (95 % CI)	Across-seed sd
DDQN	0.2105 [0.2050, 0.2155]	0.0104
WorkloadAwareThreshold	0.1943 [0.1923, 0.1964]	0.0020
AlwaysCompact_C128	0.1903 [0.1901, 0.1904]	0.0002
AttentivePPO	0.1823 [0.1669, 0.1963]	0.0403
MLP-PPO	0.1675 [0.1579, 0.1772]	0.0254

DDQN significantly outperforms every other policy (all Holm-corrected $p \leq 0.0018$). Against the best heuristic: $\Delta = +0.0162$, $p = 0.001$, Cliff’s $\delta = +0.71$ (large), and **all five seeds individually beat it** (0.198–0.224).

The reviewer’s concern was justified in a stronger sense than stated. Seed variance is large for the attention architecture (sd = 0.0403): three of its five seeds beat the best heuristic while two failed badly. A single-checkpoint evaluation — our original protocol — could therefore have reported almost any ranking. We now say this explicitly in the Discussion.

We also identified a deeper problem implied by the comment: the original Table 2 reported **step-level mean $\pm$ s.d. pooled over 5,000 steps**. That standard deviation measures within-episode variation and treats autocorrelated steps as independent samples; it is not an uncertainty estimate for a mean. All intervals are now computed over independent episodes, with across-seed variability reported separately.

Comment R1.2 — Training and inference overhead. *Wall-clock training times, GPU requirements, per-step inference latencies? Do the cloud cost savings still hold when accounting for the RL infrastructure?*

Response: Accepted. New subsection *Computational cost* plus a full hyperparameter appendix (learning rates, batch sizes, epochs, discount factors, network sizes, window sizes, seeds).

Training (per deployable agent = both specialists, mean over 5 seeds). We report hardware per row and state explicitly that wall-clock is **not** comparable across hardware:

Variant	Hardware	Params	Per agent	All 5 seeds
DDQN	Tesla T4	193,739	2.26 min	11.3 min
MLP-PPO	Tesla T4	192,203	1.73 min	8.7 min
AttentivePPO-clip	local CPU	170,571	3.30 min	16.5 min
AttentivePPO-AWR (published recipe)	Tesla T4	170,571	2.52 min	12.6 min

Training is inexpensive in absolute terms under every variant, and no GPU is required for deployment.

Per-step inference (median / p95 over 10,000 iterations, CPU only):

Component	Median	p95	Parameters
Meta-controller (rule-based)	0.0019 ms	0.003 ms	—
MLP-PPO full hierarchical step	3.48 ms	4.06 ms	192,203
DDQN full hierarchical step	4.80 ms	6.28 ms	193,739
AttentivePPO full hierarchical step	10.73 ms	12.57 ms	170,571

The rule-based meta-controller is effectively free ($\sim 2 \mu\text{s}$). Note that **AttentivePPO is $3\times$ slower than MLP-PPO despite having fewer parameters**, because self-attention is quadratic in window length — a deployment consideration parameter counts do not reveal.

Does the saving survive? Yes. Regenerated Table 11 adds an inference term (at \$0.096/h CPU) and amortised training (at \$0.526/h GPU over 100 deployment episodes):

Policy	Operational	Inference	Training	Total	vs no-maintenance
Threshold10_C128	8.37	—	—	8.37	−83.2 %
DDQN	8.79	0.0001	0.0002	8.79	−82.4 %
No_Maintenance	49.86	—	—	49.86	0 %

The original −82.9 % becomes −82.4 %; RL infrastructure is **0.003 %** of total cost. At ~ 5 ms of inference against ~ 344 ms of mean query latency, the policy costs ~ 1.4 % of a single query.

However, we now state plainly what this analysis also shows: RL is *not* the cheapest policy. **Threshold10_C128** costs less (\$8.37 vs \$8.79) because it compacts less often (124 vs 173 operations per episode). **RL buys reward — better pruning, lower latency, adaptivity — not minimum dollar cost.** This is now in both the Results and the Limitations (see also R2.16).

Comment R1.3 — Generalization beyond synthetic workloads. *Have you conducted experiments on (a) gradually drifting workloads, (b) mixed or concurrent query profiles, or (c) a different table schema? If not, discuss why results are likely or unlikely to transfer.*

Response: Accepted for (a) and (b); (c) not attempted, and we say so.

We built two genuinely held-out workloads and re-evaluated every policy:

- **v5_drift** — the same five profiles in the same order, but transitions are **linear interpolations** of query-type probabilities over 50-step windows instead of abrupt switches. The agents were trained only on abrupt shifts.
- **v5_mixed** — each phase samples **50/50 from two profiles simultaneously**, so no single query type dominates. This directly attacks the partition specialist’s mechanism, which has no single correct answer under a mixture.

Policy	Baseline	drift	mixed
DDQN	0.2105	0.2009 (−4.5 %)	0.1994 (−5.3 %)
WorkloadAwareThreshold	0.1943	0.1869 (−3.8 %)	0.1834 (−5.6 %)
AlwaysCompact_C128	0.1903	0.1902 (−0.05 %)	0.1898 (−0.2 %)
AttentivePPO	0.1823	0.1788 (−1.9 %)	0.1638 (−10.2 %)
MLP-PPO	0.1675	0.1609 (−4.0 %)	0.1469 (−12.3 %)

The RL advantage persists. DDQN remains top on both, and its margin over the best heuristic is essentially unchanged (+0.0162 in distribution, +0.0140 drift, +0.0160 mixed). Degradation tracks pruning loss almost exactly, and **AlwaysCompact_C128** — which never partitions and so has no workload-alignment mechanism to disrupt — is the **only** policy that does not degrade, confirming the effect is partition-alignment specific rather than general noise.

(c) Schema variation was not attempted. Changing row width or adding nested types requires modifying the table schema, the ingestion generator and all five query templates, which would make results non-comparable with everything else reported. We now discuss transfer explicitly in Limitations: the compaction signal (file count, block utilisation) is schema-independent and should transfer; the partition signal depends on **cardinality** of the partitioning column, so a schema with substantially different cardinality would change the pruning achievable and therefore the partition specialist’s value. We state that this is argued, not demonstrated.

Honesty about statistical power: the generalization study uses one episode per seed. With $n = 5$ paired observations, an exact two-sided permutation test has a minimum achievable p of 0.0625 — above 0.05 before correction. This study is therefore powered for **estimation, not hypothesis testing**, and we present it with confidence intervals and effect sizes ($\delta = +0.44$ to $+0.76$) rather than p -values.

Comment R1.4 — Rule-based versus learned meta-controller. *Did you experiment with a learned meta-controller? If not, can you quantify an upper bound on the performance loss attributable to the rule-based design, perhaps by comparing against an oracle?*

Response: We implemented the oracle. The result was not what we expected, and we report it as a negative finding.

We built an oracle using **environment snapshot/rollback**: at each step the world is advanced once by ingestion, a snapshot taken, and each of the three delegations (NOOP, compaction specialist, partition specialist) executed as a trial branch with its **realised** reward measured by actually running the query. The Iceberg data snapshot, partition-spec evolution, simulator state and both agents’ observation windows are then restored, and the best branch is executed for real. We chose rollback over a model-based lookahead because the reward depends on measured query latency, which no model in our system predicts (we fitted one: $R^2 = 0.67$), so a lookahead oracle would bound the *model* rather than the environment.

The greedy oracle scores **0.0546** against the rule-based controller’s **0.2306** — it does not upper-bound the design at all. The reason is measurable:

Quantity	Value
Median per-step spread between the three branch rewards	0.0069
Reward noise from query-latency variation alone (σ)	0.0120
Oracle’s gain over always-NOOP	+0.00016 per step
Correlation(branch trial reward, realised reward)	0.156

The selection signal lies below the environment’s own measurement noise, so the oracle largely selects whichever branch drew a fast query. More fundamentally, maintenance is an investment whose payoff is delayed, and one-step realised reward cannot see it. Agreement between the two controllers is 22.7%: they solve genuinely different problems.

We regard this as an informative result rather than a failed experiment: it is **evidence that “when to act” is not a greedy optimisation problem**, and that the cooldowns and urgency thresholds encode temporal structure a myopic optimiser cannot discover.

The practical bound the reviewer asked for comes from our sensitivity analysis instead: varying the meta-controller constants one at a time, the best configuration scores 0.2200 against the default’s 0.2033. **The loss attributable to suboptimal meta-controller constants is therefore $\approx 8\%$** , not the order-of-magnitude gap a naive reading of the oracle number would suggest.

We did **not** implement a learned meta-controller (option-critic or a third RL agent). We now state this as the clearest direction for future work, and note that the oracle result suggests such an agent would need a multi-step objective rather than a greedy one.

Comment R1.5 — Comparison with commercial auto-optimization. *How does your approach compare to Databricks / Snowflake / BigQuery built-in auto-optimization? If direct comparison is impossible, provide a qualitative analysis of architectural differences.*

Response: Accepted; addressed qualitatively. Direct benchmarking is not possible: these systems are proprietary, their optimisers are not separable from their execution engines, and their behaviour cannot be reproduced against our Iceberg-on-Spark environment. We add a subsection to the Discussion covering, based on publicly documented behaviour:

- **Trigger mechanism.** Commercial auto-optimisation is predominantly *threshold- and statistics-driven* — file-size and file-count targets, and for clustering, measured deviation from a target ordering. Our contribution is not a better threshold but the framing of maintenance as a **sequential decision problem under a single objective**.
- **Coupling of the two decisions.** Compaction/file-management and clustering/partitioning are typically maintained as **separate subsystems** with independent triggers. Our ablation shows why coupling matters: removing compaction collapses the *partition* benefit (pruning falls from 0.225 to 0.080) because data is never rewritten into the new layout. The interaction is non-additive, and a design that optimises the two independently cannot exploit it.
- **Layout change versus data reorganisation.** Snowflake’s micro-partitions and BigQuery’s clustering reorganise data within a fixed physical scheme; our action space includes changing the **partition specification itself**, which is a different class of decision.
- **Objective.** Commercial systems generally optimise a proxy (file size, clustering depth). Our agents optimise a composite reward spanning latency, file count, pruning and utilisation — which is both a strength (explicit trade-off) and a weakness (the weights are ours to justify; see R2.8).

We are explicit that this is a **qualitative architectural comparison, not a performance claim**, and that a fair empirical comparison would require running an identical workload on each platform — which we identify as valuable future work rather than something we have done.

Comment R1.6 — DDQN instability and off-policy adaptation. *Have you investigated prioritized experience replay, smaller target-network update frequencies, or buffers that discard old transitions? Would these mitigate the collapse, or is non-stationarity fundamentally incompatible with off-policy value-based methods here?*

Response: We investigated — and found that the collapse does not reproduce. It was an implementation defect in our code, not a property of off-policy learning.

Auditing the online path in response to this comment, we found **three defects**, each independently sufficient to produce the reported collapse:

1. **The Double-DQN update regressed the model’s softmax output, not Q-values.** Our model returns (`softmax(Q)`, `max(Q)`) for API uniformity, and the online update consumed the first return value as if it were Q . TD targets were therefore fitted against a probability vector bounded in $[0, 1]$ on the same scale as the rewards.
2. **ϵ never decayed.** It was decremented once per *update call* rather than per environment step, so “adaptive” DDQN explored at $\epsilon \approx 1.0$ throughout — acting essentially at random within whichever specialist was delegated.
3. The adaptive runs were initialised from checkpoints affected by the pipeline defects described in the Summary above.

With the update corrected to use raw Q -values and a working ϵ schedule:

Architecture	Episode 1 $\rightarrow$ 5	95 % CI	Original claim
DDQN	+3.8 %	$[-4.8, +11.4]$	-22.2 % (collapse)
MLP-PPO	-4.2 %	$[-12.9, +2.4]$	+4.9 %
AttentivePPO	+8.4 %	$[-5.0, +21.9]$	+20.3 %

All three architectures are stable; every CI straddles zero. We therefore did **not** run the remedy sweep (PER, sliding-window replay, target-update frequency): there is no collapse to rescue, and testing remedies against a non-existent failure would produce an uninformative null result.

Three claims from the original paper are withdrawn: that DDQN destabilises under adaptation; the on-policy/off-policy distinction drawn from it; and the recommendation that online adaptation be restricted to on-policy architectures. We regret that these were reported, and we note that DDQN is now the **best-performing** architecture in our study.

Honest limitation: with five 500-step episodes, each specialist receives ~ 500 transitions and ~ 25 gradient updates. This measures *fine-tuning drift*, not substantial online learning, which is why all intervals straddle zero. The defensible claim is that adaptation neither helps nor harms materially over a short deployment window — not that these architectures cannot learn online.

Comment R1.7 — Action space granularity. *Do you foresee a combinatorial explosion if Z-ordering, per-partition file size targets, and partial compaction are added, or could the hierarchical structure scale via action branching or continuous control?*

Response: Accepted as a discussion point; no new experiments. We add an *Extensibility* subsection making three arguments:

- **The hierarchy is the mechanism that prevents explosion.** The meta-controller factorises the problem into *when to act* and *what to do*, and specialists own disjoint sub-spaces. Adding a Z-ordering specialist adds one delegation target and one action set, not a Cartesian product — growth is additive in the number of specialists, not multiplicative.
- **Within a specialist, finer granularity is better handled by branching or continuous control than by enumeration.** A per-partition file-size target is naturally continuous; our discrete $\{32, 64, 128\}$ KB is a coarse discretisation of it, and our own results show target choice matters (C128 reaches 0.171 versus C32’s 0.028 on global reward). A continuous head with a bounded output would remove the discretisation without enlarging the action count.
- **Partial compaction is the genuinely hard case**, because it requires selecting a *subset* of files — combinatorial in the number of files. We suggest this is better expressed as a parameterised action (a predicate over file size and age) than as a set-valued action, and we flag it as the direction most likely to require a different formulation rather than a larger action space.

We are explicit that these are design arguments, not results.

Reviewer #2

Comment R2.1 — No significance testing. *The central claims of superiority rest on no significance testing at all, which is critical given the top three agents differ by ~ 0.0005 – 0.0077 against a per-step std of ~ 0.064 .*

Response: Accepted. See R1.1 for the full protocol. The reviewer’s specific observation is exactly right and we now address it directly: the ~ 0.064 figure is a **step-level** standard deviation,

and comparing mean differences against it — or reporting it as an uncertainty — conflates within-episode variation with uncertainty in the mean. All statistics are now computed at the episode level across independent training runs.

Comment R2.2 — “+4.3 % over the best heuristic” not defensible. *The flagship claim is well within one standard deviation and $n = 5$ episodes, so “outperform 17 baselines” is not defensible without CIs or a paired test.*

Response: Accepted; the claim as written has been removed.

It is now: **DDQN outperforms the best heuristic by +0.0162 (+8.3 %), paired permutation $p = 0.001$, Cliff’s $\delta = +0.71$ (large), with all five independent training runs individually beating it.** The comparison is paired on 25 matched episodes.

Equally important, the *class-level* claim does **not** hold and has been removed: MLP-PPO now ranks **below three heuristics** and is significantly worse than the best heuristic (-0.0268 , $p = 0.0018$). The paper no longer claims that RL as a class outperforms the baselines; it claims that **the best RL configuration does**.

Comment R2.3 — Three-way RL comparison undermines any ranking. *The authors report a -0.2 % gap between the two best agents; either add statistical tests or reframe all three as statistically indistinguishable.*

Response: Accepted, and the corrected results give a sharper answer than either option offered. After correcting the pipeline defects, the three architectures are *not* uniformly indistinguishable:

Comparison	Δ	Holm p	Cliff’s δ	Verdict
DDQN vs AttentivePPO	+0.0282	0.0018	+0.44 medium	significant
DDQN vs MLP-PPO	+0.0430	0.00105	+0.84 large	significant
AttentivePPO vs MLP-PPO	+0.0148	0.31	+0.33 medium	not significant

DDQN is significantly ahead of both; **AttentivePPO and MLP-PPO are statistically indistinguishable**, exactly as the reviewer anticipated for that pair. We also report a second dimension the original paper omitted: **across-seed stability** (DDQN 0.0104, MLP-PPO 0.0254, AttentivePPO 0.0403), which separates the architectures as sharply as mean performance does.

Comment R2.4 — External validity; simulation-only. *The evaluation is entirely simulation-based with synthetic data, so external validity to a real query engine is untested despite the “backed by a real Iceberg table” framing.*

Response: Accepted. The framing was imprecise and has been corrected throughout. What is real: an actual Apache Iceberg table on Apache Spark with a REST catalog and S3-compatible object storage; genuine Parquet files; genuine `rewrite_data_files` compaction; genuine partition-spec evolution; and **measured** query latency. What is not real: the scale (tables reach $\sim 4\text{--}5$ MB, not TB), the data (synthetic), the query set (five templates), and the pruning benefit (modelled — see R2.5).

We now state in the Abstract and Limitations that this is a **simulation environment backed by a real table engine at micro-scale**, and that external validity to production-scale deployments is untested. We also note a specific consequence: because query latency at this scale is dominated by per-file overhead rather than data volume, the latency–file-count relationship is likely steeper than in production.

Comment R2.5 — Pruning matrix is a hardcoded assumption. *Because query benefit is driven entirely by a fixed lookup table, the “workload-aware” advantage is partly baked into the environment design rather than discovered; justify these values empirically or test sensitivity.*

Response: Accepted — and this is the criticism we consider most consequential. We have tested sensitivity; we have **not** measured the values empirically, and we now say so plainly rather than implying they are grounded.

Sensitivity analysis: all non-zero pruning values scaled by $\{0.8, 0.9, 1.0, 1.1, 1.2\}$, with both the best RL agent and the best heuristic re-evaluated at each scale so the *gap* can be tracked rather than only absolute scores:

Scale	DDQN	WorkloadAwareThreshold	Gap
0.8	0.1865	0.1817	+0.0048
0.9	0.1976	0.1882	+0.0094
1.0	0.2005	0.1971	+0.0034
1.1	0.2122	0.2015	+0.0106
1.2	0.2162	0.2076	+0.0086

The RL advantage is positive at every scale, including a 20 % weaker pruning benefit. We also note a fact that partly answers the “baked in” concern directly: **DDQN wins while achieving *lower* pruning than the best heuristic** (0.204 vs 0.248). Its advantage comes from compaction quality — 37 files versus 50, 220 ms versus 324 ms — not from exploiting the pruning table more aggressively.

We agree that the honest resolution is empirical measurement (instrumenting Spark’s scan metrics to record files-skipped per query) and we state this as required future work. We do not claim the current values are validated.

Comment R2.6 — 2×-compressed workload is near in-distribution. *Training on v5-standard and evaluating on a 2×-compressed version of the same distribution is close to in-distribution testing.*

Response: Accepted. See R1.3: two genuinely held-out workloads (drifting transitions, concurrent mixed profiles) were built and all policies re-evaluated. The RL advantage persists (DDQN margin +0.0140 drift, +0.0160 mixed, versus +0.0162 in distribution).

We also note for completeness that policy rankings on the compressed and full workloads agree at Spearman $\rho = 0.964$ — which supports using the compressed protocol for *efficiency*, while confirming the reviewer’s point that it does **not** constitute a generalization test.

Comment R2.7 — The coordination layer is hand-tuned; constants receive no sensitivity analysis. *The “when-to-act” decision is not learned, weakening the “deep RL framework” framing, and none of $\theta = 0.35$, +0.15 boost, 4/20-step cooldowns receive a sensitivity analysis.*

Response: Both points accepted.

On framing: we have rewritten the description throughout. The system is a **hierarchical framework with a rule-based arbitrator and two learned specialists**; the “when to act” decision is not learned, and the paper now says so in the Abstract, Methods and Discussion rather than implying end-to-end learning.

On sensitivity: a one-at-a-time sweep around every constant ($\theta \in \{0.25 \dots 0.45\}$, boost $\in \{0, 0.15, 0.30\}$, compaction cooldown $\in \{2, 4, 8\}$, partition cooldown $\in \{10, 20, 40\}$), each configuration evaluated on the same three matched episodes:

Total spread across the entire grid: 0.0295 (14.5 % of the default). The published constants are a reasonable but **mildly conservative** operating point — a shorter compaction cooldown (2 instead of 4) would have improved reward by 8.2 %. Performance degrades gracefully; no configuration collapses.

We report two honest caveats: (i) with three episodes per configuration the sweep is powered for **estimation, not inference** (an exact permutation test on $n = 3$ cannot reach $p < 0.25$), so we present magnitudes rather than p -values; and (ii) the two worst configurations fall slightly **below** the best heuristic, so the meta-controller is robust *within a sensible range*, not universally.

Comment R2.8 — Reward weights are magic numbers; train/eval objective mismatch. *The reward weights are fixed magic numbers with no justification or ablation, and agents are trained on compact/partition rewards while the paper evaluates on a separately-weighted global reward — a train/eval mismatch never analyzed.*

Response: Both accepted. The second point had not occurred to us and is correct.

Weight sensitivity. Each of the four global-reward weights was perturbed $\pm 25\%$ and all four renormalised; the global reward was recomputed per step from logged metrics and policies re-ranked:

Variant	Spearman vs published	Top policy	Rank changes
latency $\pm 25\%$	1.000	DDQN	0
files $\pm 25\%$	1.000	DDQN	0
utilisation $\pm 25\%$	0.983 / 0.950	DDQN	2 / 3
pruning $\pm 25\%$	0.950	DDQN	3

The ranking is stable (Spearman ≥ 0.95 , DDQN top under all nine weightings); the changes that occur are adjacent swaps among mid-table heuristics.

On the train/eval mismatch, we now analyse it explicitly rather than leaving it implicit. The specialists are trained on `compact_reward` and `partition_reward` while evaluation uses `global_reward`, and the objectives are **not** aligned in one specific and consequential way: the compaction reward contains a hand-designed *target-size bonus* that favours `COMPACT_32KB` at low ingestion, whereas measured global reward strongly favours larger targets (C128 0.171, C64 0.120, C32 0.028). We show this is not hypothetical — our MLP-PPO agent selected C32 for 99 of its 103 compactions, which is a direct consequence of the mismatch. This is now reported as a design flaw in the reward decomposition and as a probable contributor to that architecture’s underperformance.

Comment R2.9 — “Closing 146.9 % of the initial gap” over-interprets 5 points. *These should be softened or supported with more episodes and variance bands.*

Response: Accepted; the claim is removed. As reported under R1.6, the adaptive experiment was invalid for three independent reasons and has been rerun. The regenerated results carry confidence intervals and show **all three architectures neutral** (CIs straddling zero). The “146.9 % of gap closed” figure does not appear in the revised manuscript; the “% gap closed” metric itself is now guarded (see R2.13).

Comment R2.10 — “+220–225 % over PartitionOnly” is near-tautological. *PartitionOnly is a degenerate baseline (uncontrolled file explosion), so the comparison is near-tautological; the informative gain is +25–29 % over CompactOnly.*

Response: Accepted. The percentage framing over `PartitionOnly` has been removed. The ablation is now presented in absolute reward with the informative comparison foregrounded:

Configuration	Reward	Δ vs full
Full hierarchical agent	0.1823	—
CompactOnly (partition specialist removed)	0.1202	−0.0621
PartitionOnly (compaction specialist removed)	−0.1099	−0.2922

We agree the **CompactOnly** comparison is the informative one, and we strengthen it: **full and CompactOnly are physically almost identical** (51.6 vs 48.4 files; 331 vs 314 ms), so the entire 0.062 difference is attributable to pruning (0.225 vs exactly 0.000). **PartitionOnly** is retained only to establish the **non-additive interaction**: its pruning collapses to 0.080 *despite being the partition-only agent*, because without compaction data is never rewritten into the new layout. That is a mechanistic point, not a headline gain.

Comment R2.11 — The attention benefit is asserted, never demonstrated; confounded with capacity. *There is no attention visualization or analysis, and the +3.5 % edge is confounded with capacity differences.*

Response: Both accepted. Neither the advantage nor the mechanism survived investigation.

Capacity. The reviewer is right that the comparison was confounded — and in a way we had not noticed, because the confound runs in **opposite directions per specialist**: MLP-PPO has 17 % *fewer* parameters on compaction but 42 % *more* on partition. We built a parameter-matched MLP (within **0.06 %**):

Model	Reward (95 % CI)	Parameters
AttentivePPO	0.1823 [0.1669, 0.1963]	170,571
Parameter-matched MLP	0.1774 [0.1664, 0.1885]	170,667

$p = 0.54$, Cliff’s $\delta = +0.16$ (small) — **not significant**. The +3.5 % advantage survives neither the unmatched ($p = 0.31$) nor the matched comparison, and the claim has been removed.

An incidental finding: the matched MLP **outperforms** the published MLP (0.1774 vs 0.1675) while using *fewer* total parameters. The published model was simply mis-allocated — starving compaction, over-provisioning partition — so part of the original gap was architecture *sizing*, not architecture family.

Mechanism. We now provide the analysis that was missing. Multi-head attention scores were captured at every step of frozen 1,000-step rollouts (3 seeds):

Steps back from newest	0	1	2	3
Compaction (uniform 0.100)	0.150	0.125	0.113	0.102
Partition (uniform 0.050)	0.517	0.105	0.051	0.032

The partition specialist places **52 % of its attention on the single newest step**; the compaction specialist is only **1.29× uniform**, close enough to flat that the global average pooling which follows performs a similar function. Attention concentrated on one position is a **recency gate**, not sequence modelling — and around the four workload transitions the change in recency focus is negligible and **inconsistent in direction across seeds** ($\Delta = -0.0066$ compaction, $+0.0031$ partition).

We therefore state that the encoder does not demonstrably attend to workload transitions, which is what “workload-aware attention” would require. This also explains the other results coherently: an MLP can express a fixed positional weighting (hence the matched tie); $7\times$

the parameters changes nothing (0.1830), so the mechanism is not capacity-limited; and doubling the window *hurts* (0.1765), because extending a static profile only adds diluting positions.

We add an exploratory tuning study (reported in full, all variants) asking whether attention improves under a configuration suited to it. Only longer training helps, and only modestly: +0.0058, $p = 0.11$ — reaching parity with the best heuristic (0.1949 vs 0.1943) but remaining below DDQN. We state explicitly that **DDQN was not given an equivalent sweep**, so this means *attention can be brought closer with tuning*, not *attention is as good*.

Comment R2.12 — No training time, compute, hyperparameters, learning rates, seeds, step counts. *This prevents assessment of whether modest reward gains justify the added complexity, especially since agents are idle > 76 % of the time.*

Response: Accepted. All of it is now reported — see R1.2 for wall-clock, hardware and inference latency, plus a complete hyperparameter appendix (learning rates, batch sizes, epochs, γ , network sizes, window sizes, entropy coefficients, clip ratios, target-update frequencies, and the seed set) and per-run manifests recording the git commit and hardware for every result.

On the complexity trade-off, we now address it directly rather than leaving it to the reader — see R2.16.

Comment R2.13 — “−3,207 % gap closed” is a division-by-near-zero artifact. *It should be replaced with a note that the metric is undefined when the gap is negligible.*

Response: Accepted exactly as proposed. The metric is now computed by a guarded function that returns “n/a” when |initial gap| falls below a threshold (0.01, chosen because the reward is $O(0.1\text{--}0.3)$ per step, so smaller gaps lie within episode noise). In the regenerated Table 7, DDQN’s entry is n/a (initial gap 0.0076). The guard is part of the released code, not a manual edit.

Comment R2.14 — Dueling formula typo; double-DDQN target never described. *The advantage-centering term is written as a subtraction of $A(s)$ rather than the mean over actions, and the “double” mechanism is never described.*

Response: The typo is confirmed and corrected; the implementation was correct. We verified the source directly: both the model registry (`v + (a - tf.reduce_mean(a, axis=-1, keepdims=True))`) and the training notebook implement the standard mean-subtraction form. Equation (6) has been corrected to

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') \right)$$

We have also added the missing description of the Double-DDQN target: the **online** network selects the next action and the **target** network evaluates it,

$$y = r + \gamma Q_{\theta}(s', \arg \max_{a'} Q_{\theta}(s', a'))(1 - d)$$

with the target network synchronised every 50 gradient steps.

In writing this description we discovered that our *online* implementation did not match it — it regressed the network’s softmax output rather than raw Q-values. This is reported under R1.6 and is the reason the DDQN adaptation results changed.

Comment R2.15 — No dedicated related-work section. *References 12–28 are listed but never discussed, and positioning against learned data-layout work (Qd-tree, SageDB) is missing.*

Response: Accepted. A dedicated **Related Work** section has been added, organised into four threads with explicit positioning:

1. **Learned data layout** — Qd-tree, SageDB, learned index structures. These learn a *layout* offline for a known query distribution; we learn a *policy* that changes layout online as the distribution shifts, and we pay ongoing maintenance cost rather than a one-time reorganisation.
2. **RL for database tuning** — QTune, LlamaTune, HUNTER, CTuner, HWTune, MLETune. These tune configuration knobs for a DBMS; the action does not rewrite data. Our actions are physically destructive rewrites with delayed, partially irreversible effects.
3. **Lakehouse storage and benchmarking** — Iceberg, Delta Lake, Hudi, LST-Bench, Dostoevsky’s merge trade-offs, which frame the problem but do not address online joint optimisation.
4. **Hierarchical and multi-agent RL** — FeUdal Networks, option-critic, and multi-agent surveys, positioning our rule-based arbitrator honestly as a *simplification* of a learned manager (see R2.7 and R1.4).

Comment R2.16 — Cost/complexity trade-off versus a strong single heuristic. *Given AlwaysCompact_C128 reaches 0.1869 and agents are idle most of the time, discuss the cost/complexity trade-off of a three-architecture neural system versus a strong single heuristic.*

Response: Accepted, and we now argue against our own system where the evidence requires it. A new *When is RL worth it?* subsection states:

- **On cost, the heuristic wins.** Threshold10_C128 is the cheapest policy in our own model (\$8.37 vs DDQN’s \$8.79), and no RL agent repays its training cost relative to it. RL buys reward, not dollars.
- **The reward advantage is real but modest:** +8.3% over the best heuristic, significant with a large effect size.
- **Idleness is a feature, not overhead.** Agents act on < 24% of steps, and the per-step cost of deciding *not* to act is ~5ms against ~344ms of query latency — 1.4%. Idleness is the correct behaviour, not wasted computation.
- **The honest recommendation:** for a static workload, a well-chosen heuristic such as AlwaysCompact_C128 or Threshold10_C128 is a strong, cheap and simple choice, and we say so. The RL system earns its complexity when the workload *shifts* — which is where partition alignment matters and where our generalization experiments show the advantage persisting.

We also note that the flat per-operation compaction price in our cost model under-charges aggressive strategies; at production scale, where a compaction is a distributed rewrite whose cost grows with bytes touched, a policy that compacts 1,000 times per episode would be penalised far more heavily. We add a size-proportional cost variant (ranking unchanged at our scale) and state the caveat explicitly.

Comment R2.17 — Repository URL truncated. *The repository URL appears truncated and should be verified as complete and resolvable.*

Response: Confirmed and corrected. We verified both forms:

URL	HTTP status
.../LakePilot-Research-prot (as printed)	404
.../LakePilot-Research-prototype	200

The statement has been re-typeset with the complete URL and now also cites the archived release’s Zenodo DOI, **10.5281/zenodo.21860273**, per the editor’s requirement.

Comment R2.18 — Cost model uses fixed per-operation prices. *A brief sensitivity analysis, or at least a caveat on how conclusions shift under different cloud pricing, would strengthen the commercial-cost claims.*

Response: Accepted; a full sweep was run rather than a caveat. Query, compaction and storage unit prices were each varied over $\{0.5\times, 1\times, 2\times\}$ — all **27 combinations**. `Threshold10_C128` is cheapest in all 27, and `No_Maintenance` is most expensive in all 27; **the cost ordering never flips**.

We additionally added a **size-proportional compaction cost** (charging by bytes rewritten, calibrated to the same mean) to test the more realistic pricing model. The ranking is essentially unchanged — one adjacent swap — because every policy compacts at a similar table size at this scale. We state clearly that the two models bracket the realistic case (flat = pure per-job overhead; size-proportional = pure bytes rewritten) and that agreement under both is a robust result *at this simulation's scale*, with production-scale divergence flagged as a limitation.

Summary table of reviewer concerns

Point	Concern	Resolution
E-1	Accurate reporting, no overstated conclusions, full limitations	CIs, paired tests and effect sizes on every claim; unsupported claims removed; expanded <i>Limitations</i>
E-2	Code deposit in a DOI-assigning repository	Zenodo deposit, DOI 10.5281/zenodo.21860273; new <i>Code availability</i> section
R1.1	Statistical validation across independent runs	Five seeds per architecture, bootstrap CIs, Wilcoxon and permutation tests, Cliff's δ , Holm correction (Table 1)
R1.2	Training and inference overhead; do savings hold?	New <i>Computational cost</i> subsection; regenerated cost table (-82.4%); RL is not the cheapest policy
R1.3	Generalization beyond synthetic workloads	Two held-out workloads (<code>v5_drift</code> , <code>v5_mixed</code>); schema variation not attempted and stated as such
R1.4	Rule-based versus learned meta-controller	Snapshot/rollback oracle implemented; reported as a negative finding; $\approx 8\%$ loss bound from the sensitivity sweep
R1.5	Comparison with commercial auto-optimization	Qualitative architectural comparison added to the Discussion; explicitly not a performance claim
R1.6	DDQN instability under adaptation	Collapse traced to three implementation defects; re-runs show all three architectures neutral; three claims withdrawn
R1.7	Action space granularity	New <i>Extensibility</i> subsection; design arguments, no new experiments
R2.1	No significance testing	Episode-level statistics across independent runs (see R1.1)
R2.2	" $+4.3\%$ over the best heuristic" not defensible	Claim removed; replaced by $+0.0162$ ($+8.3\%$), $p = 0.001$, $\delta = +0.71$; class-level claim dropped
R2.3	Three-way ranking undermined	DDQN significantly ahead of both; AttentivePPO and MLP-PPO indistinguishable; across-seed stability added
R2.4	External validity; simulation-only	Framing corrected throughout to a simulation backed by a real engine at micro-scale
R2.5	Pruning matrix is a hardcoded assumption	Sensitivity sweep over $\{0.8-1.2\}$; advantage positive at every scale; empirical measurement stated as required future work
R2.6	$2\times$ -compressed workload is near in-distribution	Genuinely held-out workloads added (see R1.3); Spearman $\rho = 0.964$ reported for the compressed protocol
R2.7	Hand-tuned coordination layer; no sensitivity analysis	Framing rewritten as a rule-based arbitrator; full one-at-a-time sweep, spread 0.0295 (14.5%)
R2.8	Reward weights; train/eval objective mismatch	$\pm 25\%$ weight perturbations (Spearman ≥ 0.95); mismatch analysed and reported as a design flaw
R2.9	" 146.9% of gap closed" over-interprets 5 points	Claim removed; experiment rerun with CIs; metric guarded (see R2.13)
R2.10	" $+220-225\%$ over PartitionOnly" near-tautological	Percentage framing removed; ablation reported in absolute reward with CompactOnly foregrounded
R2.11	Attention benefit asserted, confounded with capacity	Parameter-matched MLP ($p = 0.54$); attention-score analysis shows a recency gate; claim removed
R2.12	No training time, compute, hyperparameters, seeds	All reported (see R1.2) plus a complete hyperparameter appendix and per-run manifests
R2.13	" $-3,207\%$ gap closed" is an artifact	Guarded metric returning "n/a" below a 0.01 gap threshold; guard released in the code
R2.14	Dueling formula typo; Double-DQN target undescribed	Equation (6) corrected; Double-DQN target added; online implementation defect found and reported
R2.15	No dedicated related-work section	New Related Work section with four positioning threads
R2.16	Cost/complexity versus a strong single heuristic	New <i>When is RL worth it?</i> subsection; the heuristic wins on cost and we say so

Closing note

Several of these comments led us to results that contradict our original submission. We have chosen to report them fully — including three withdrawn claims and two defects in our own pipeline — rather than present a narrower revision that would have been easier to defend. We believe the resulting paper is more useful: its central claim is now backed by a significance test with a large effect size and consistency across five independent training runs, and its negative results (why greedy meta-control fails, why offline RL on heuristic logs leaks trajectory identity) are transferable beyond this application.